Sorting :- Arranging data in increasing / decreasing order based on a parameter

e.g. 3 8 9 14 19 } → based on value in increasing order

e.g. 19 14 9 8 3 } → based on value in decreasing order

e.g. 1 5 3 9 6 10 12 } → data is sorted on the basis of # factors

e.g.	1	5	3	9	6	10	12
# factors :-	1	2	2	3	4	4	6

→ sort() function in your choice of language

→ T.C. for sort fn → O (N log N)

Quest Minimum Cost to Remove all Elements

Given N array elements, at every step remove an array element.

Cost to remove an array element = sum of array elements present in array

Find min. cost to remove all elements.

e.g. 1 :- {2, 1, 4}

		Cost
Remove 2:	{2, 1, 4} :-	7
Remove 1 :-	{1, 4} :-	5
Remove 4 :-	{4} :-	4
	Cost :-	16

		Cost
Remove 4:	{2, 1, 4} :-	7
Remove 2 :-	{1, 2} :-	3
Remove 1 :-	{1} :-	1
	Cost :-	11

Quiz1 {4, 6, 1}

	cost
Remove 6 →	11
Remove 4 →	5
Remove 1 →	1
	17

Quiz2 {~~3~~, ~~5~~, ~~1~~, -3}

	Cost
Remove 5 →	6
Remove 3 →	1
Remove 1 →	-2
Remove -3 →	-3
	total = 2

e.g. {a, b, c, d}

		Cost
Remove a :-	{a, b, c, d}	a + b + c + d
Remove b :-	{b, c, d}	b + c + d
Remove c :-	{c, d}	c + d
Remove d :-	{d}	d

total cost :- a + 2b + 3c + 4d → Minimum cost

$a + 2b + 3c + 4d$

4 highest coefficient

1st element = 1st max

2nd ele = 2nd max

3rd ele = 3rd max

4th elem = 4th max

Pseudo Code:-

1. Sort in descending order
2. sum = 0 (Initialise)

```
for( i=0; i<N; i++){
    sum= sum + arr[i]( i+1 )
}
```

T.C = O(N log N + N)

T.C = O(N log N)

Dry run:-

{2, 1, 4}

Sort:- {4, 2, 1}

sum = 4*(1) + 2*(2) + 1*(3)

= 4 + 4 + 3

= 11

Q Noble Integer

Given N array elements, calculate the number of noble integers present.

arr[i] is said to be noble if (No. of elements < arr[i]) == arr[i]

e.g. { 1, -5, 3, 5, -10, 4 } → Ans = 3

#cnt :- 2 1 3 5 0 4

-10, -5, 1, 3, 4, 5

Quiz3 { -3, 0, 2, 5 } → Ans = 1

0 1 2 3

Quiz4 { -10, -5, 1, 3, 4, 5, 10 } Ans = 3

0 1 2 3 4 5 6

Brute force:-

For every element, get cnt of elements smaller than arr[i] and then compare.

T.C = $O(N^2)$

S.C = $O(1)$

Optimised solution

1) Sort array in ascending order

2) ans = 0

```
for(i=0; i<N; i++){
    if(arr[i]==i){
        ans++;
    }
}
return ans;
```

T.C = $O(N \cdot \log N + N)$

$= O(N \log N)$

Q3 Noble Integer:- Elements can repeat

e.g. { 0 2 2 4 4 6 } → Ans = 1

cnt :- 0 1 1 3 3 5

Quiz5 { -10, 1, 1, 3, 100 } → Ans=3

cnt:- 0 1 1 3 4

e.g. { -10, 1, 1, 1, 4, 4, 4, 7, 10 } → ans = 7

0 1 1 1 4 4 4 7 8

Brute force will 100% work here → TODO

e.g. { 0 2 2 4 4 6 }

cnt:-	0	1	1	3	3	5
index:-	0	1	2	3	4	5

→ Different

Quiz6 { -10, 1, 1, 2, 4, 4, 4, 8, 10 } → Ans = 5

cnt < arr[i] :- 0 1 1 3 4 4 4 7 8

Quiz7 { -3, 0, 2, 2, 5, 5, 5, 5, 8, 8, 10, 10, 10, 14 } → Ans = 7

cnt:-	0	1	2	2	4	4	4	4	8	8	10	10	10	13
index:-	0	1	2	3	4	5	6	7	8	9	10	11	12	13

Ans = 0, 1, 2, 3, 4, 5, 6, 7.

Observations:-

1) Cnt is same when elements are same
2) Whenever we come to the first occurence of an ele, the count is same as index.

no. of elements < arr[i] = i

Pseudo code:-

arr[i] != arr[i-1]

```
sort arr[] in asc order
int ans=0;
int less=0;
if(arr[0]==0) ans++;
for(i=1; i<N; i++){
    if(arr[i-1] != arr[i]){
        less = i;
    }
    if(less == arr[i]){
        ans++;
    }
}
```

T.C = O(N log N)
S.C = O(1)

Break → 10:45 PM

Comparator

Q Given N array elements, sort them in increasing order of their no. of factors. If 2 elements have same no. of factors, element with less value should come first.

Note:- No extra space

e.g. {9, 3, 4, 8, 16, 37, 6, 13, 15}

#factors = 3, 2, 3, 4, 5, 2, 4, 2, 4

o/p:- 3, 13, 37, 4, 9, 6, 8, 15, 16

e.g. {1, 21, 6, 23, 10, 14, 25}

factors :- 1, 4, 4, 2, 4, 4, 3

o/p:- 1, 23, 25, 6, 10, 14, 21

→ No extra space

→ You know in-built sort() func.

→ Modify the in-built sort () function using a comparator.

e.g. 25 → 3   16 → 5   → 25 will come first

e.g. 10 → 4   9 → 3   → 9 will come first

e.g. 49 → 3   25 → 3   → 25 will come first.

In Java,

```
Arrays.sort(arr[], comp)

bool comp(int a, int b) {
    Note:- If you want a to come before b in the final sorted order,
           then your function should return True

    int f1 = factors(a);
    int f2 = factors(b);
    if (f1 < f2) { return True };
    else if (f1 == f2  &&  a < b) { return True; }
    else { return false };
}

sort(arr[], comp);
```

We will learn internals of this in the next 2 months.

T.C. → We will discuss.

Q Given N array elements, sort them in inc order of no. of digits in the number.
If 2 elements have same no. of digits, element with more value should come first.

eg.

93	2	37	639	8	100	345	49
↓	↓	↓	↓	↓	↓	↓	↓
2	1	2	3	1	3	3	2

O/p:- 8, 2, 93, 49, 37, 639, 345, 100

```
bool comp (int a, int b) {
    int d1 = digits (a);
    int d2 = digits (b);
    if (d1 < d2) { return True }
    else if (d1 == d2 && a > b) { return True }
    else return False;
}

Sort (arr[], comp);
```

TODO:- check syntax of comparator func. in your language

Note:- If you want a to come before b in the final sorted order, then your function should return True else return False

```
digits (x) {
    return you no. of digits in x
}
```